UNIVERSITÀ DI TORINO

Hybrid Workflows for Artificial Intelligence

Hybrid workflows

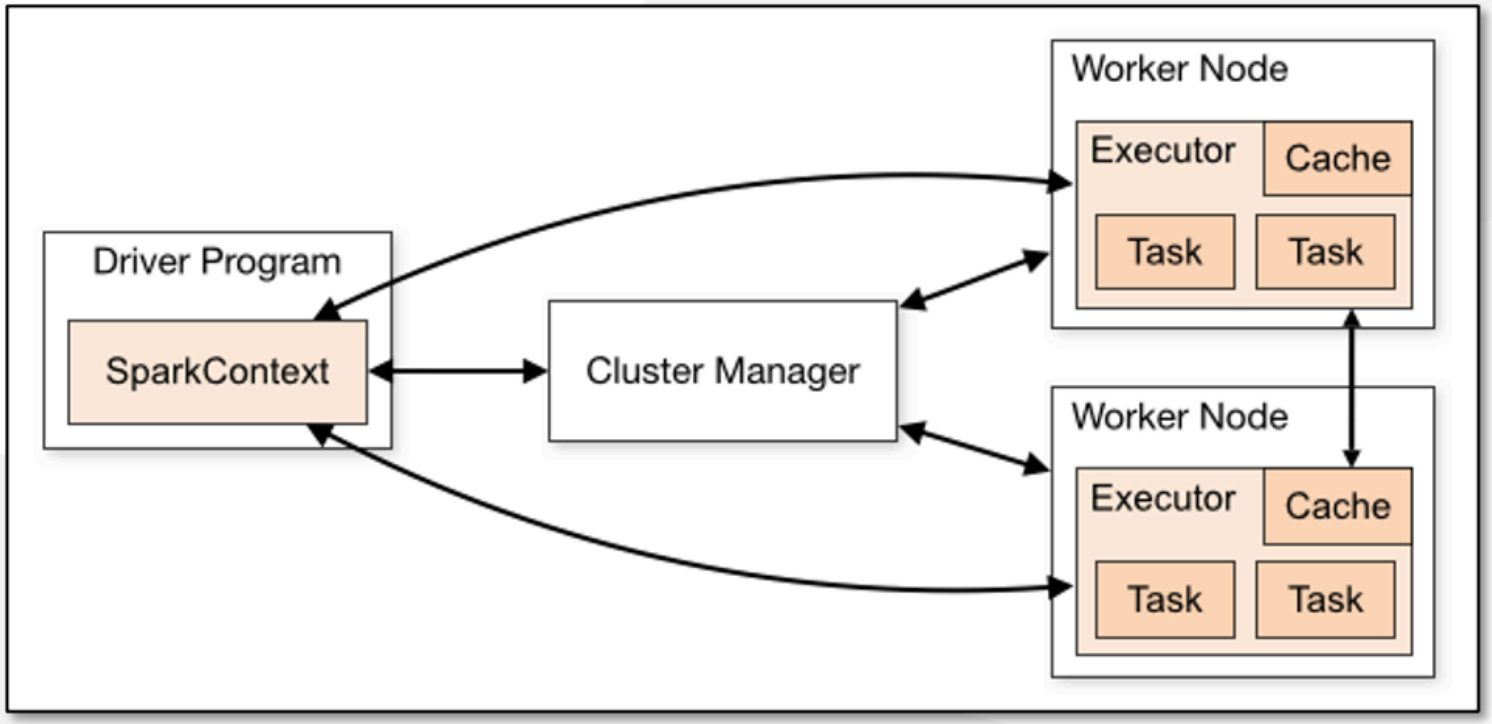
Prof. Iacopo Colonnelli

iacopo.colonnelli@unito.it

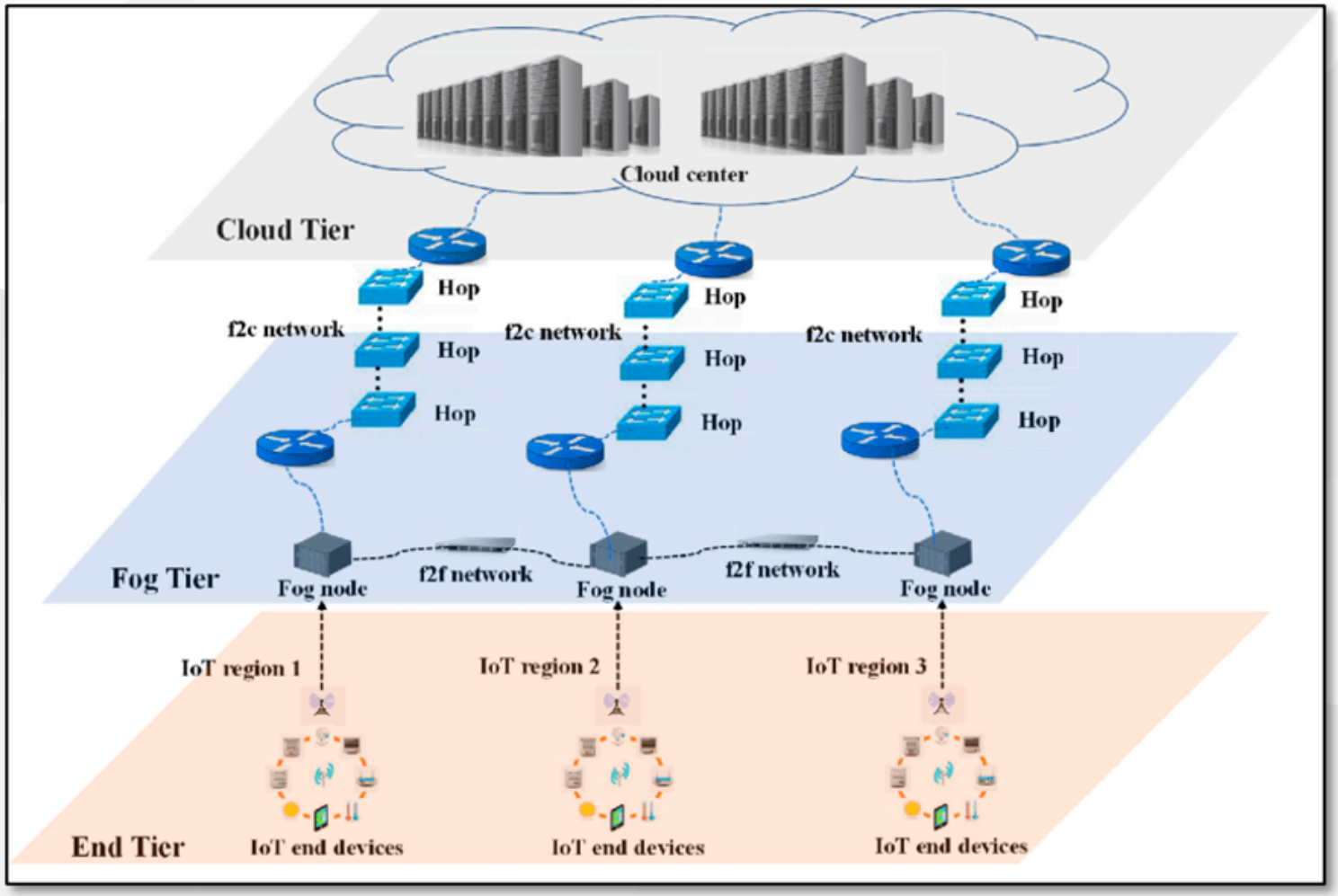
https://alpha.di.unito.it/iacopo-colonnelli

Challenges in modern scientific workflows

Each step of a distributed application can require **multiple intercommunicating agents** (e.g., a Spark cluster or a microservices architecture)



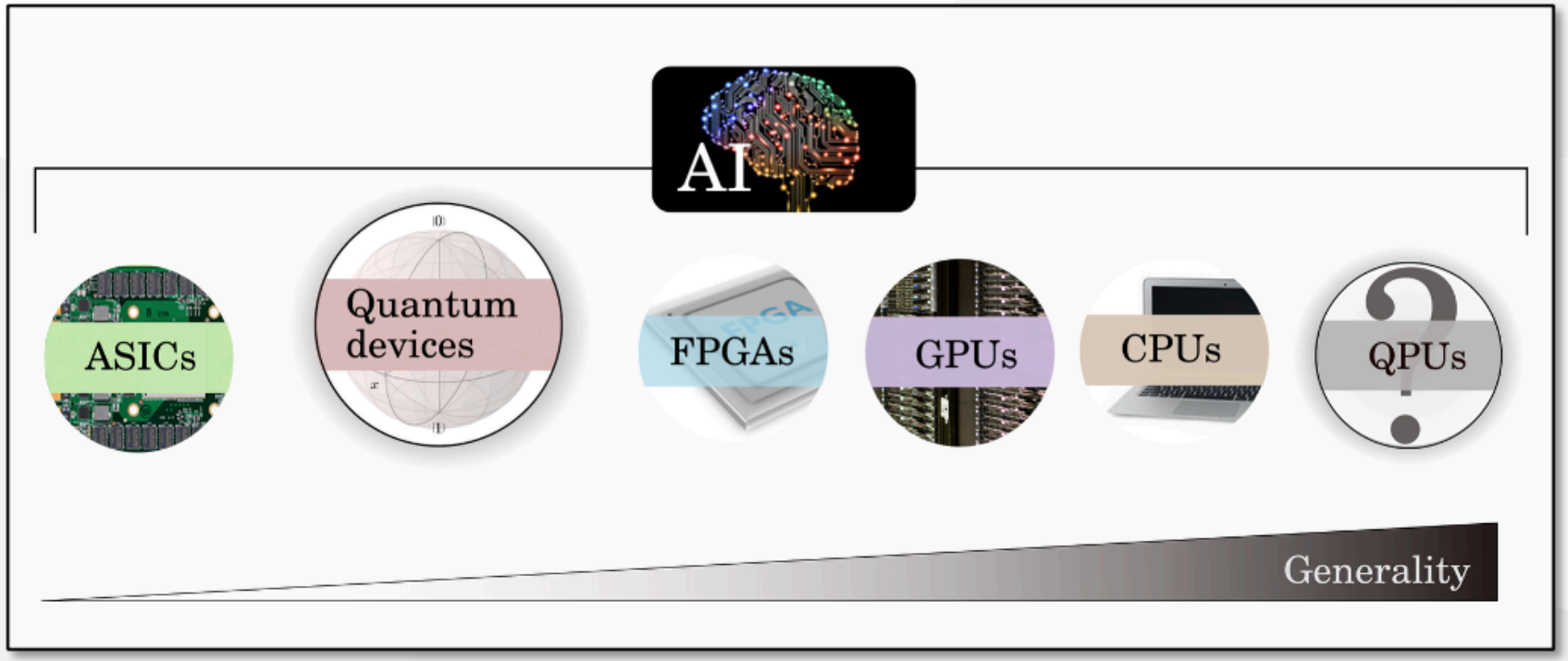
Challenges in modern scientific workflows



Large-scale architectures can be **modular**, and modules can be **independent** of each other (e.g., modular HPC and infrastructure federations)

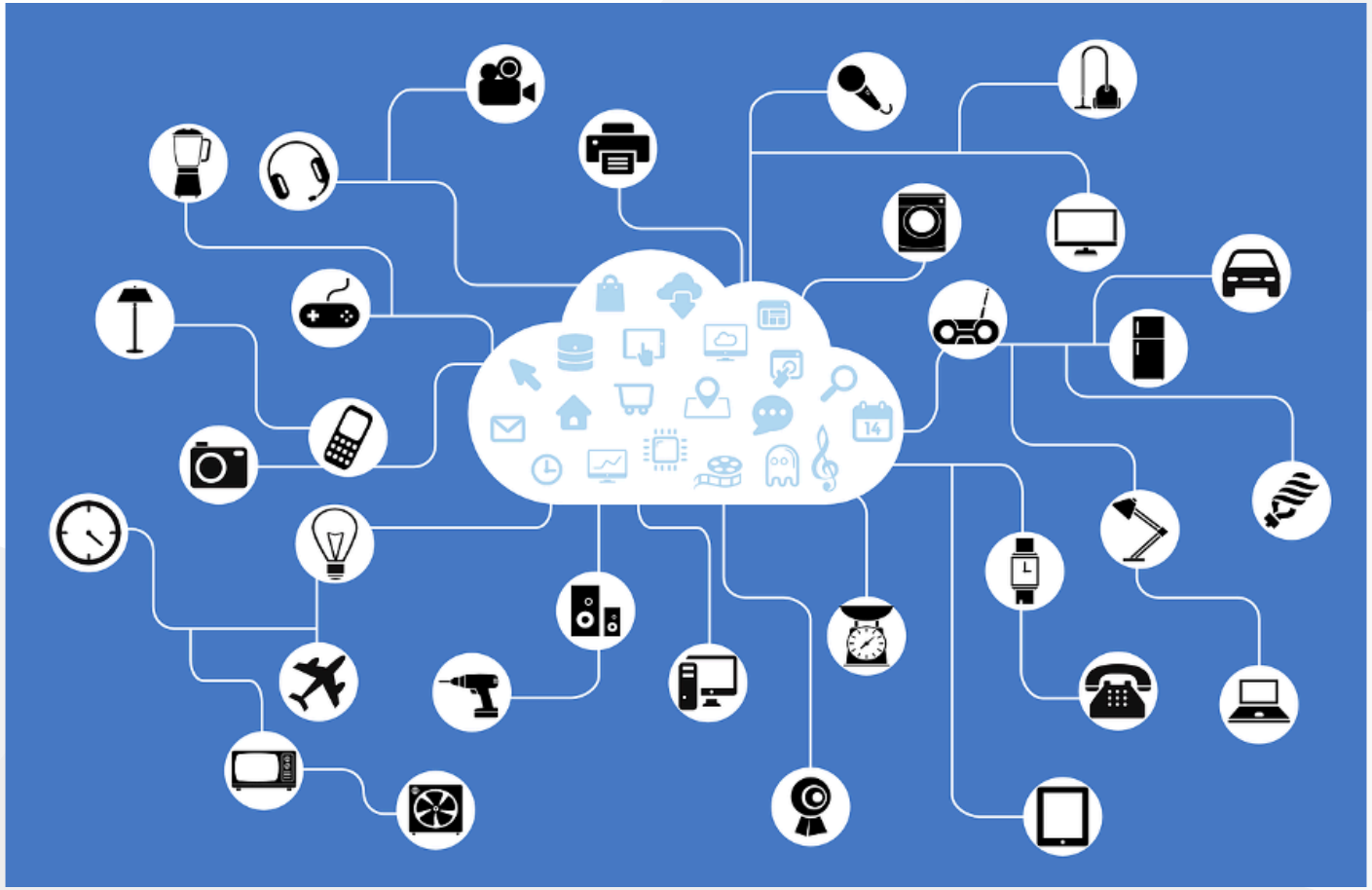
Challenges in modern scientific workflows

Large-scale architectures can be **heterogeneous** (e.g., Cloud+HPC environments and Classical+Quantum computing)

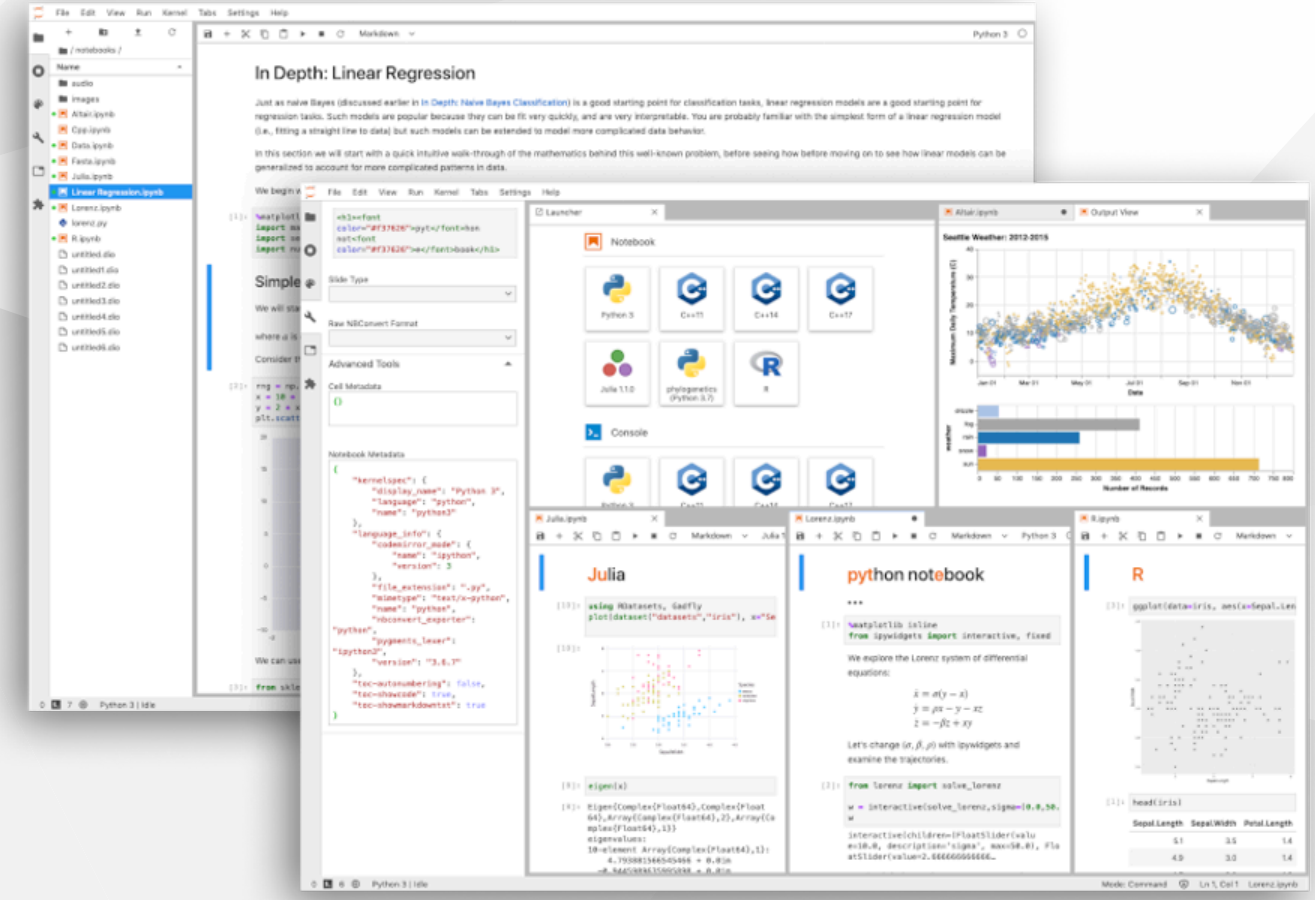


Challenges in modern scientific workflows

Large-scale architectures can be **fully distributed** (e.g., Edge and Fog computing), without the possibility to support a centralised control plane for workflow orchestration



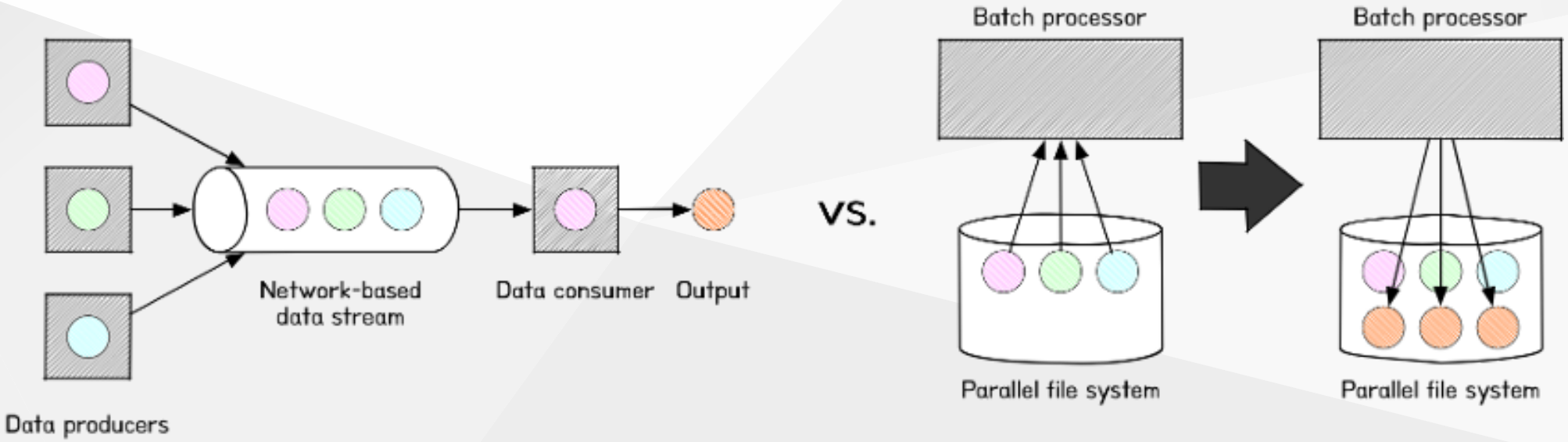
Challenges in modern scientific workflows



Even in the continuum, WMSs may need to support **runtime intervention** for observability and early-stopping of large-scale workflows (e.g., through Jupyter Notebooks)

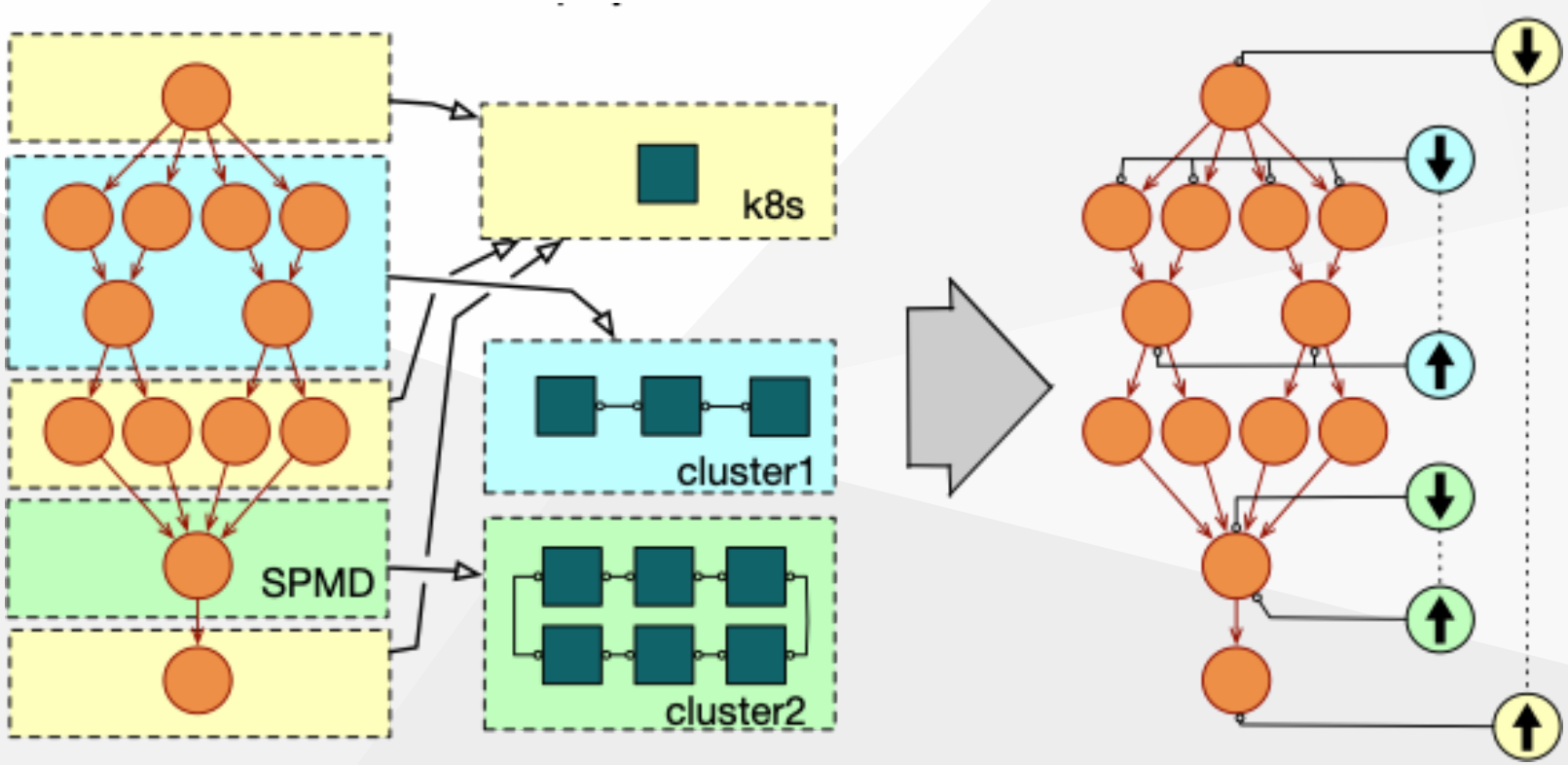
Challenges in modern scientific workflows

Continuum workflows must deal with **different data management strategies** (e.g., streaming at the edge vs. batched on Cloud/HPC) and different **data transport media** (e.g., network-based at the edge vs. file-based on data centres)



Hybrid workflows

A **hybrid workflow** is defined as a workflow whose steps can be distributed on **multiple**, **heterogeneous**, and **independent** computing infrastructures



Hybrid workflows

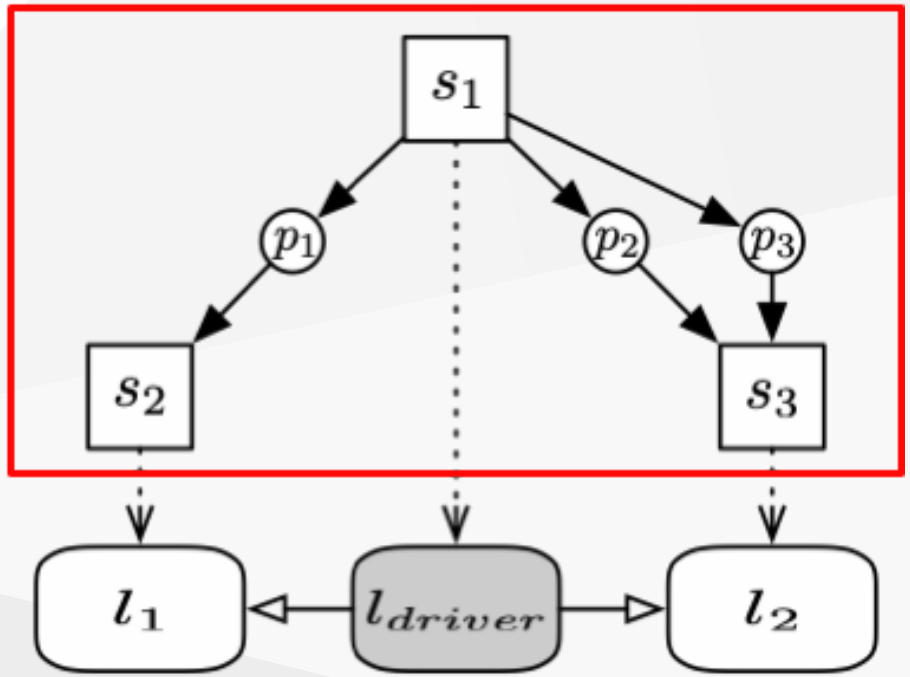
A **hybrid workflow** is defined as a workflow whose steps can be distributed on **multiple**, **heterogeneous**, and **independent** computing infrastructures

- Support for **multiple** infrastructures implies that each step must potentially target a different **deployment location** in charge of executing it
- Locations can be **heterogeneous**, exposing **different methods and protocols** for authentication, communication, resource allocation and job execution
- Plus, they can be **independent** of each other, meaning that they **may not be allowed to communicate directly** for message passing and data transfer operations

Hybrid workflows

In practice, a hybrid workflow model comprises:

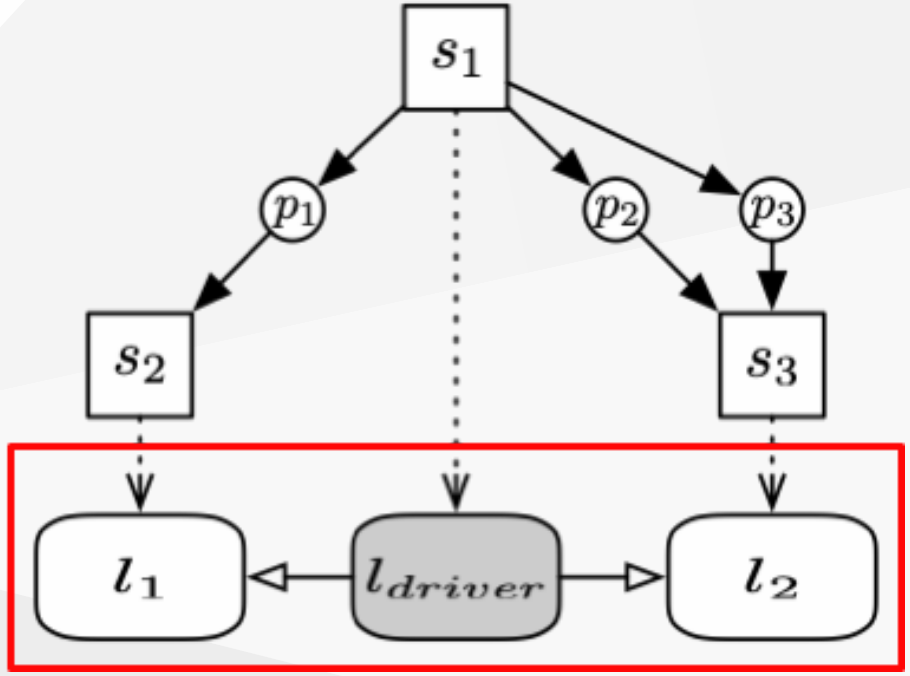
- A **workflow model**, i.e., a directed bipartite graph encoding executable steps, data ports and dependencies between them



Hybrid workflows

In practice, a hybrid workflow model comprises:

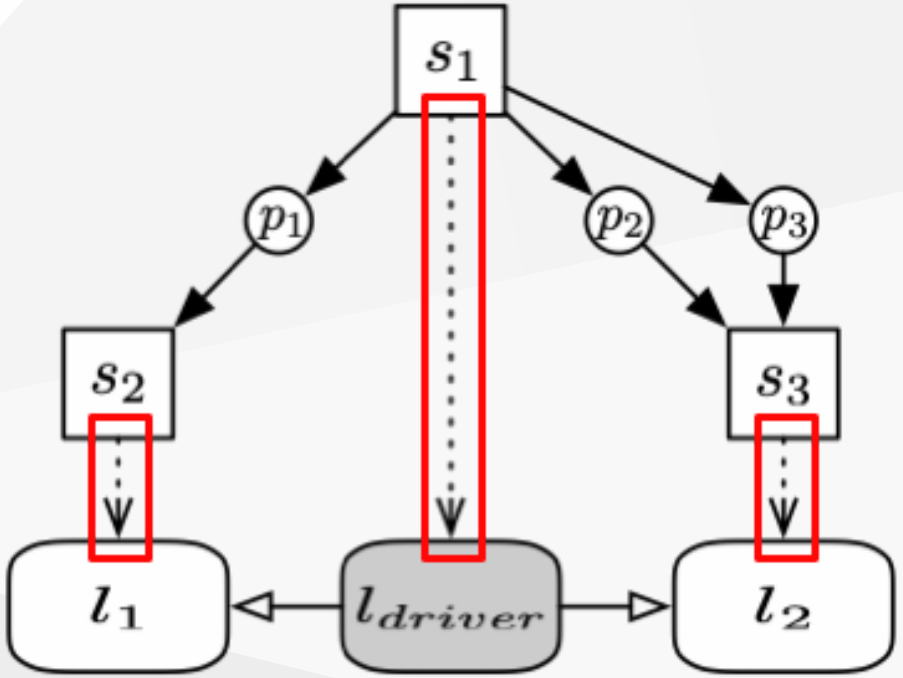
- A **workflow model**, i.e., a directed bipartite graph encoding executable steps, data ports and dependencies between them
- A **topology of deployment locations**, i.e., a directed graph where the nodes are the locations in charge of executing steps and the links are directed communication channels between locations



Hybrid workflows

In practice, a hybrid workflow model comprises:

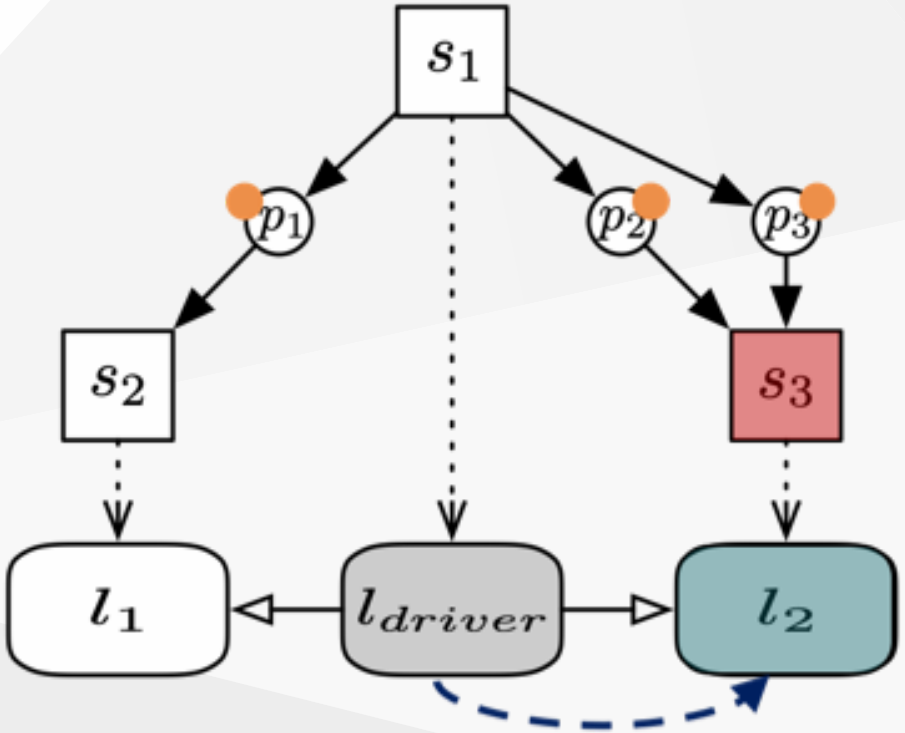
- A **workflow model**, i.e., a directed bipartite graph encoding executable steps, data ports and dependencies between them
- A **topology of deployment locations**, i.e., a directed graph where the nodes are the locations in charge of executing steps and the links are directed communication channels between locations
- A **binding relation**, i.e., a many-to-many relation stating which locations are in charge of executing each workflow step



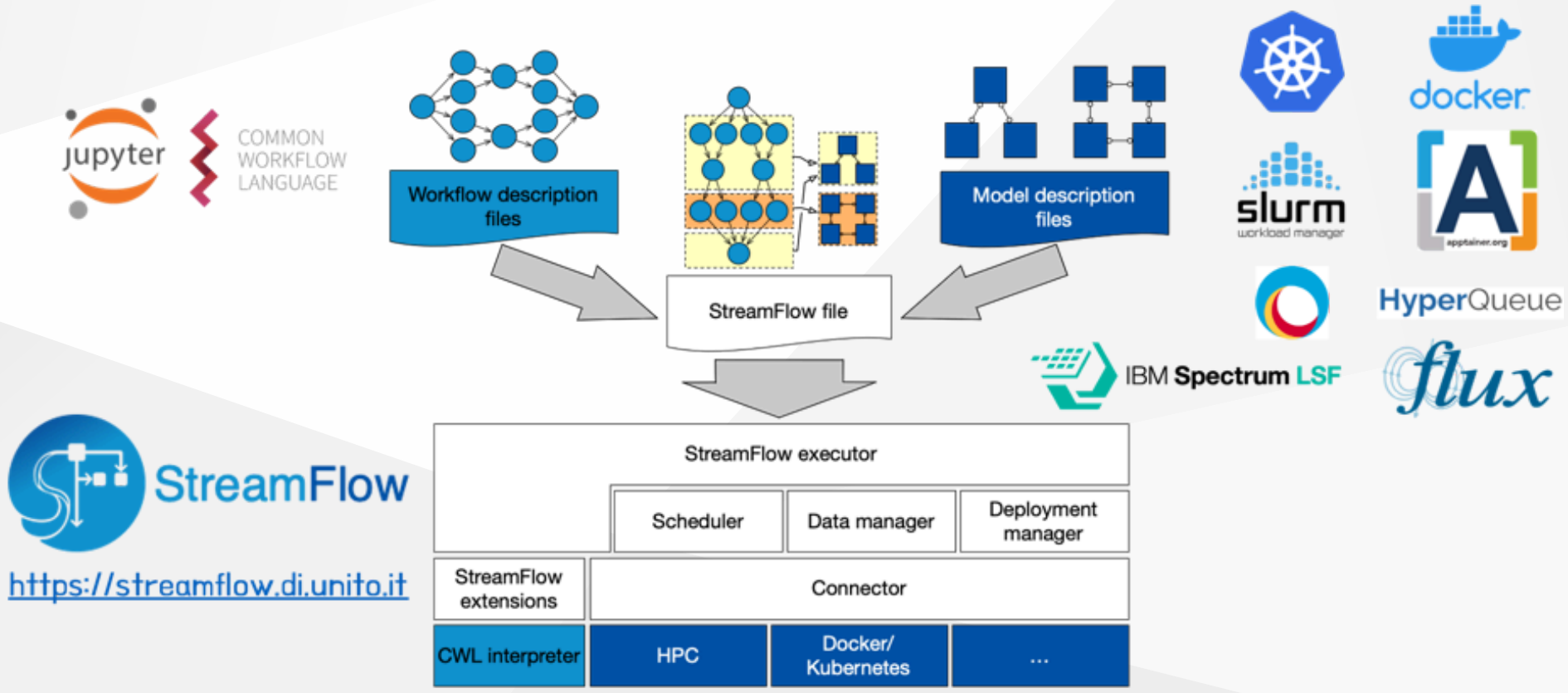
Hybrid workflows

A step s becomes **fireable** (i.e., ready for execution) when:

- Each input port $In(s)$ contains enough **tokens**
- Its related location is **deployed**
- All its input data have been **transferred** on that location



The StreamFlow WMS



I. Colonnelli, B. Cantalupo, I. Merelli and M. Aldinucci, "StreamFlow: cross-breeding cloud with HPC," in IEEE Transactions on Emerging Topics in Computing, vol. 9, iss. 4, p. 1723-1737, 2021. doi:10.1109/TETC.2020.3019202

The StreamFlow file

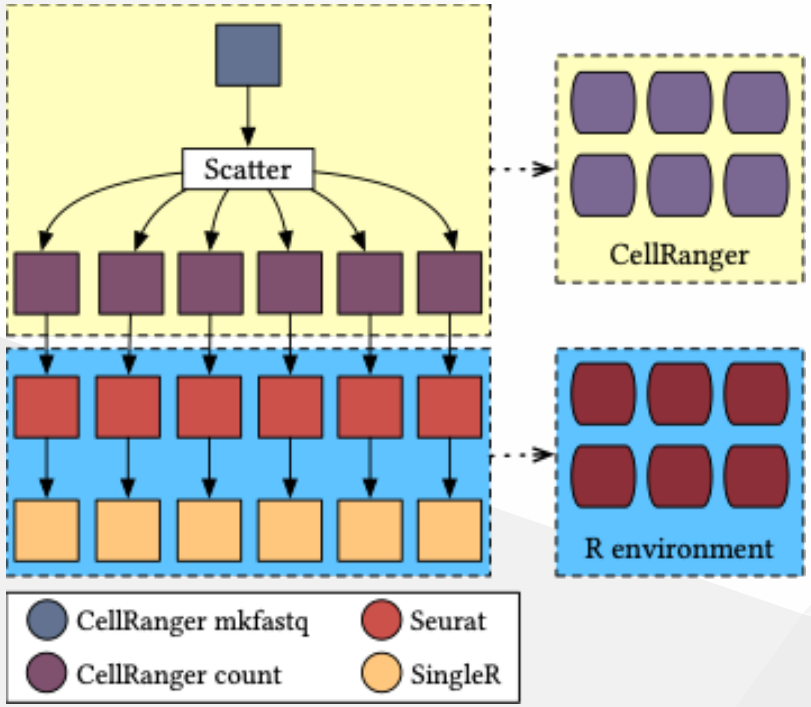
The `streamflow.yml` file maps each step or sub-workflow of a hybrid workflow onto one or more (local or remote) execution locations in charge of its execution

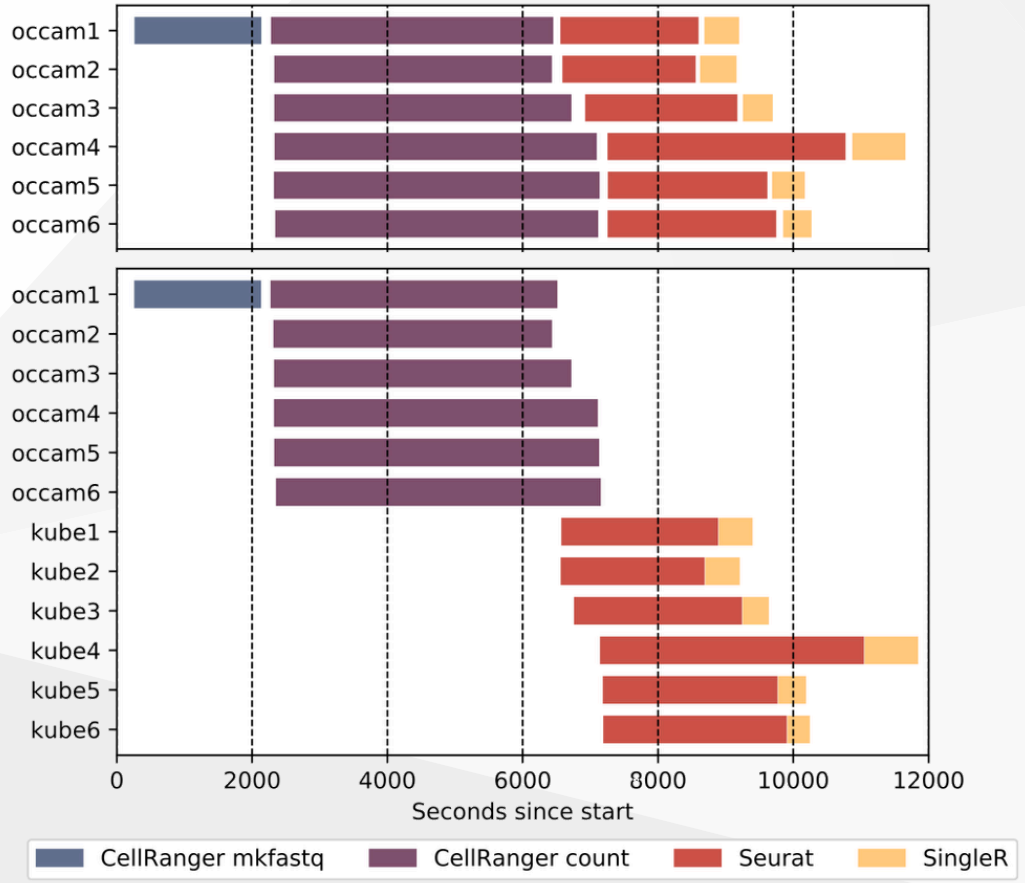
StreamFlow follows a **fully declarative approach**, fostering portability and reproducibility

```yaml
version: v1.0
workflows:
  extract-and-compile:
    type: cwl
    config:
      file: main.cwl
      settings: config.yml
    bindings:
      - step: /compile
        target:
          deployment: docker-openjdk

deployments:
  docker-openjdk:
    type: docker
    config:
      image: openjdk:9.0.1-11-slim
```

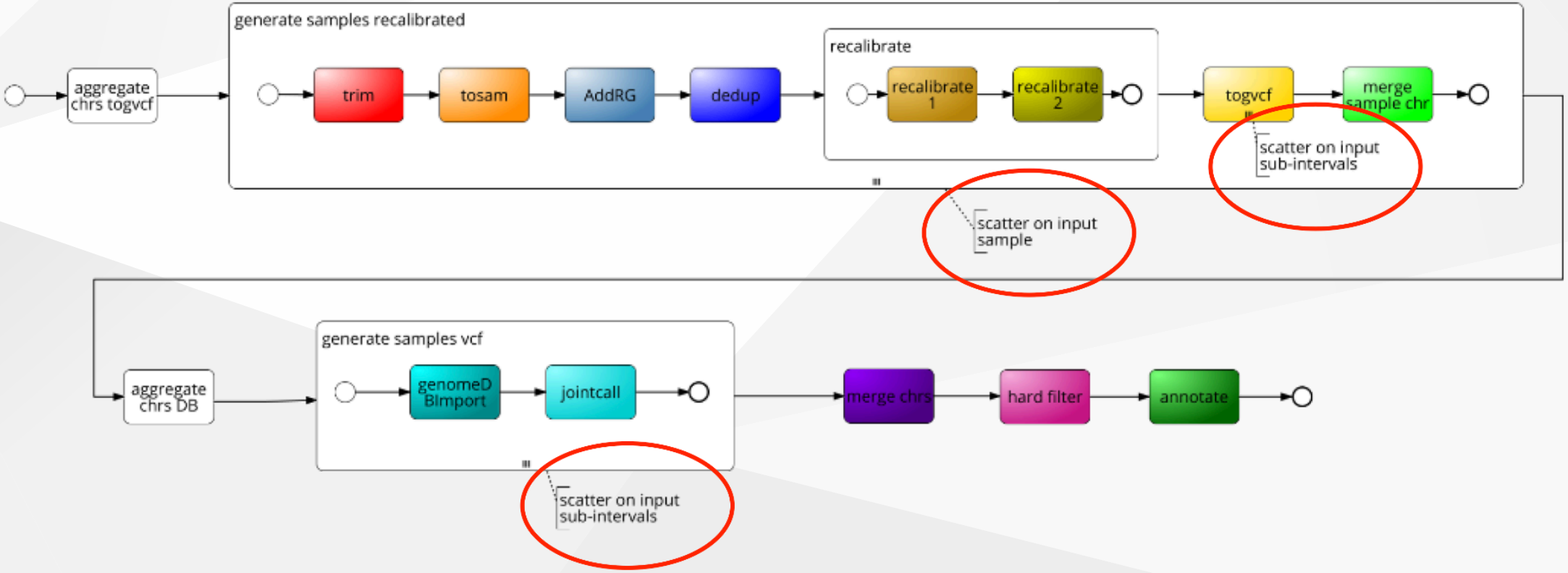
Case study: single-cell pipeline





I. Colonnelli, B. Cantalupo, I. Merelli and M. Aldinucci, "StreamFlow: cross-breeding cloud with HPC," in IEEE Transactions on Emerging Topics in Computing, vol. 9, iss. 4, p. 1723-1737, 2021. doi:10.1109/TETC.2020.3019202

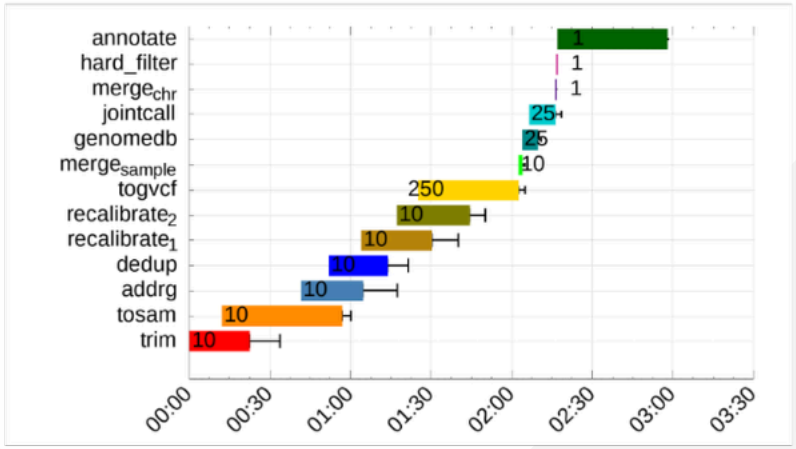
Case study: variant calling pipeline

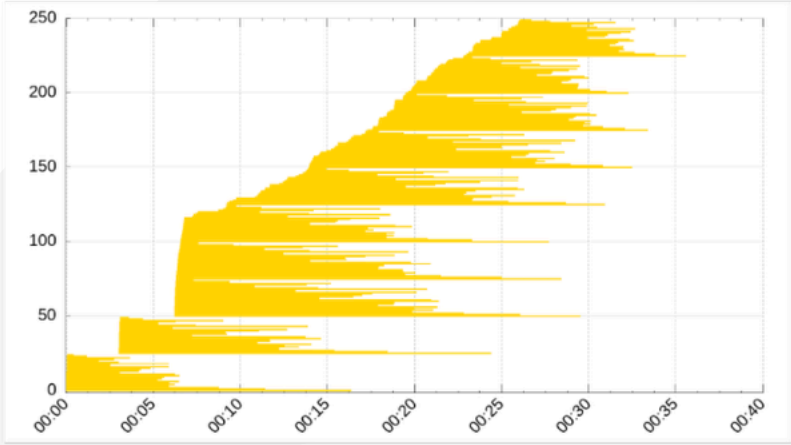


A. Mulone, S. Awad, D. Chiarugi, and M. Aldinucci, "Porting the variant calling pipeline for NGS data in cloud-HPC environment," in 47th IEEE annual computers, software, and applications conference, COMPSAC 2023, Torino, Italy, 2023, p. 1858–1863. doi:10.1109/COMPSAC57700.2023.00288

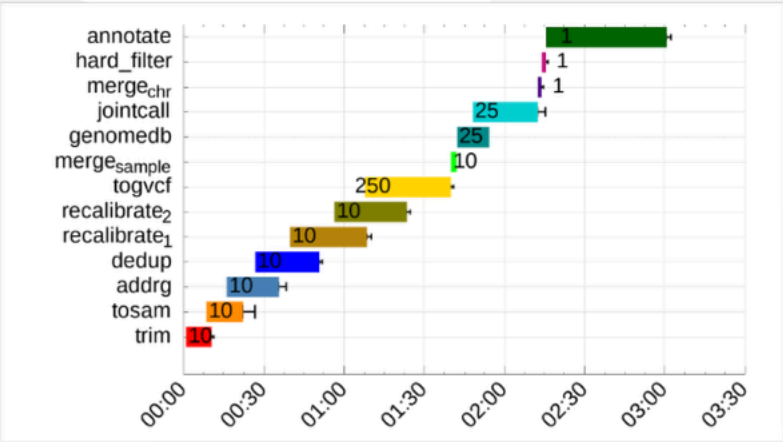
Case study: variant calling pipeline

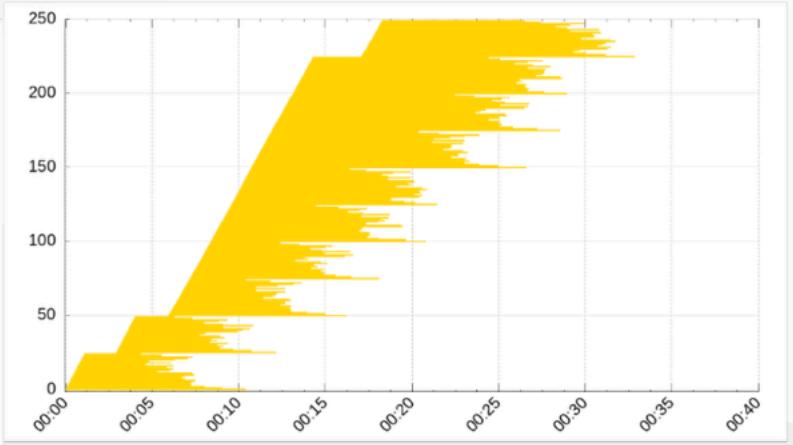
Cloud VM
(96 cores)





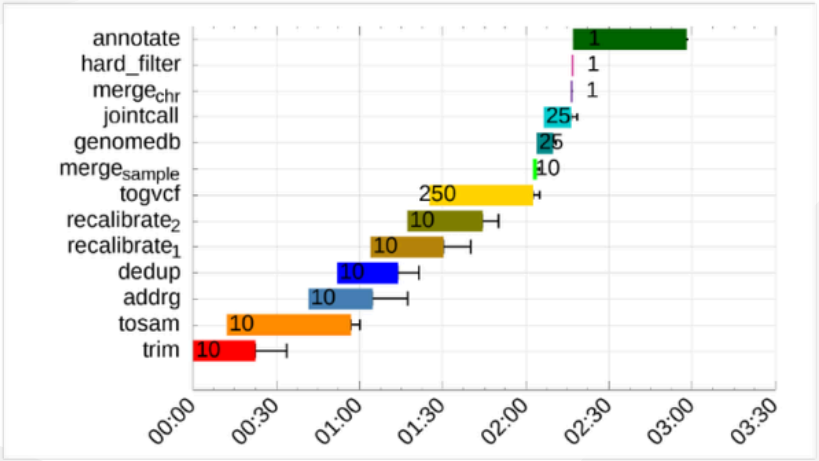
HPC system
(CINECA Galileo100)

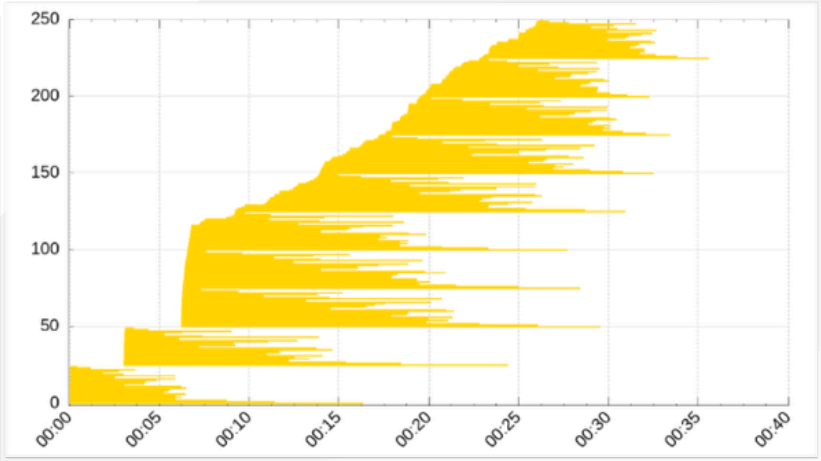




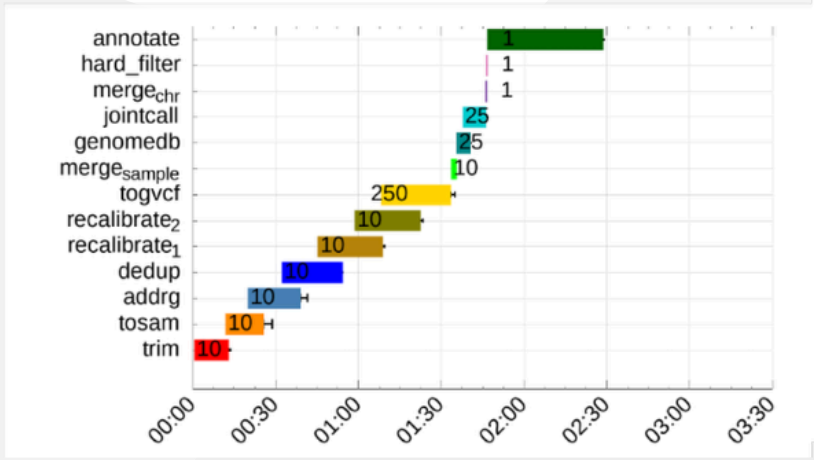
Case study: variant calling pipeline

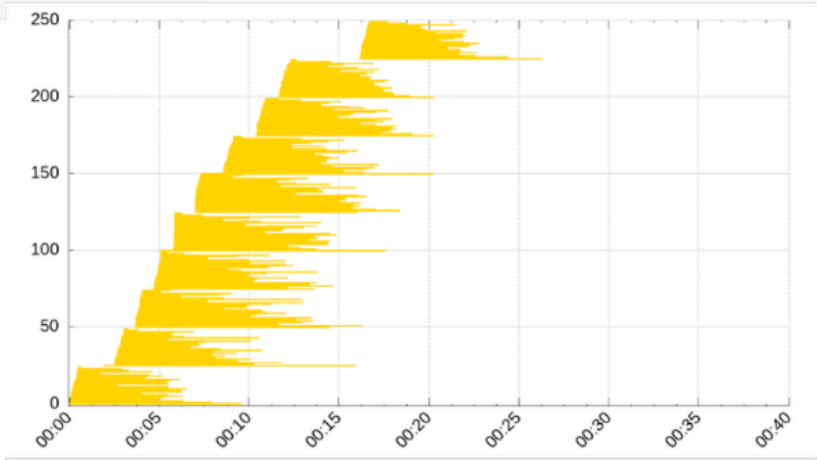
Cloud VM
(96 cores)



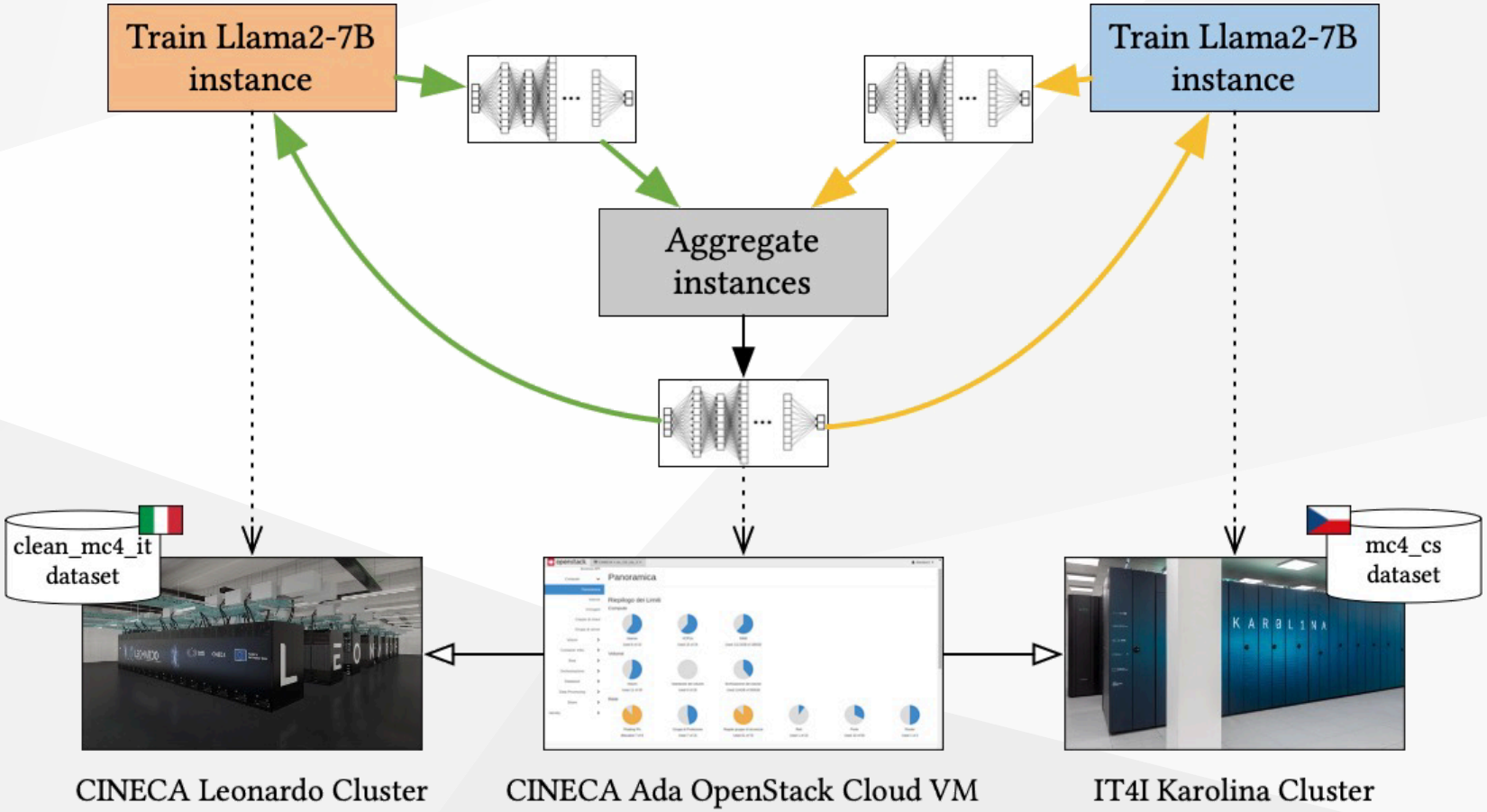


Hybrid
(Cloud + HPC)





Case study: cross-facility Federated Learning



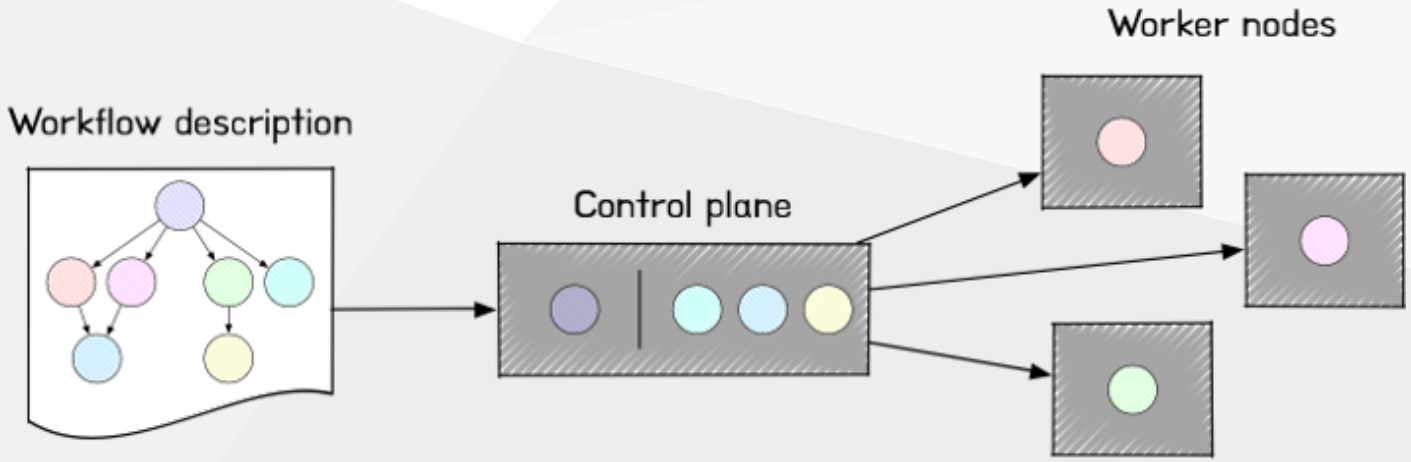
I. Colonnelli, R. Birke, G. Malenza, G. Mittone, A. Mulone, J. Galjaard, L. Y. Chen, S. Bassini, G. Scipione, J. Martinovič, V. Vondrák, and M. Aldinucci, "Cross-Facility Federated Learning," in Procedia Computer Science, vol. 240, p. 3–12, 2024. doi:10.1016/j.procs.2024.07.003

Decentralised workflows

Workflow languages are designed to be **managed by humans**: their translation to distributed execution plans is normally quite complex

Workflow runtimes usually require a **central control plane** that orchestrates workflow execution (or delegates resource management to a different central unit)

Serverless workflows are apparently decentralised, but in reality they just delegate the execution to a third-party infrastructure, which commonly relies on **centralised units** (function registry, resource manager, ...)

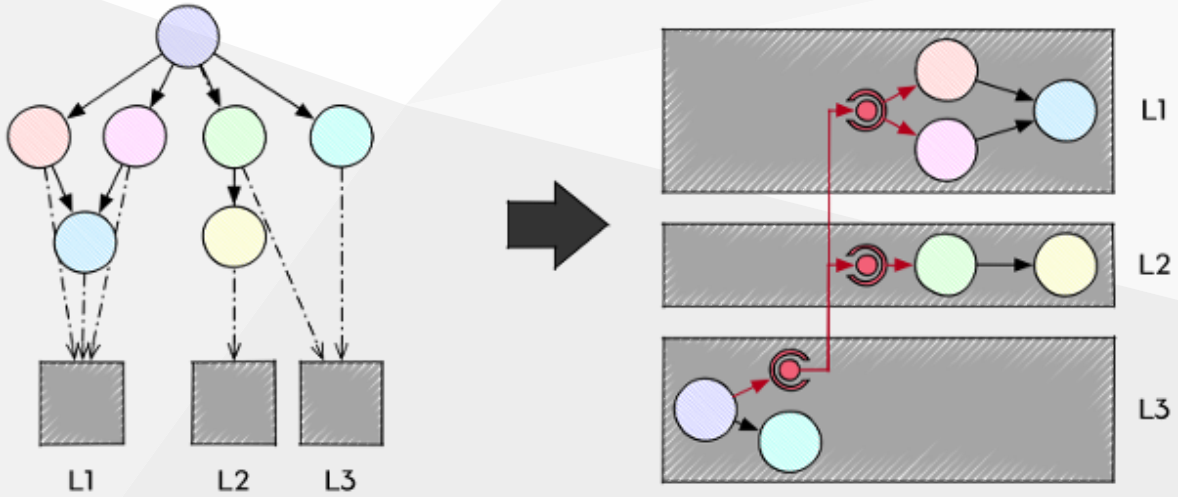


SWIRL: a low-level workflow representation

The **Scientific Workflow Intermediate Representation Language (SWIRL)** represents a distributed workflow as a low-level, machine-oriented representation language

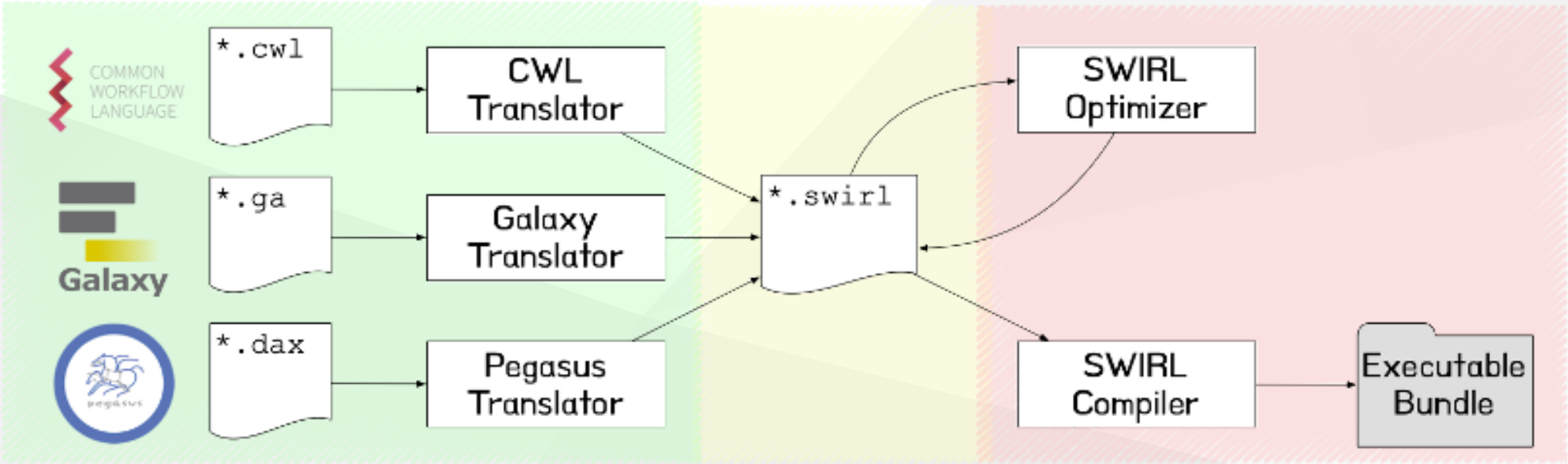
SWIRL can project a single workflow model into a **distributed execution plan**, where locations communicate by means of **send/receive primitives**

SWIRL also **decouples** the workflow description from the orchestration strategy, allowing for **transparent adaptation** to different runtime platforms (e.g, from batched HPC to cloud FaaS)



The SWIRL compiler

The **swirlc** compiler translates a high-level hybrid workflow representation written by a user (e.g., CWL, Galaxy or Pegasus) into a **low-level distributed executable bundle** optimized for a specific execution environment



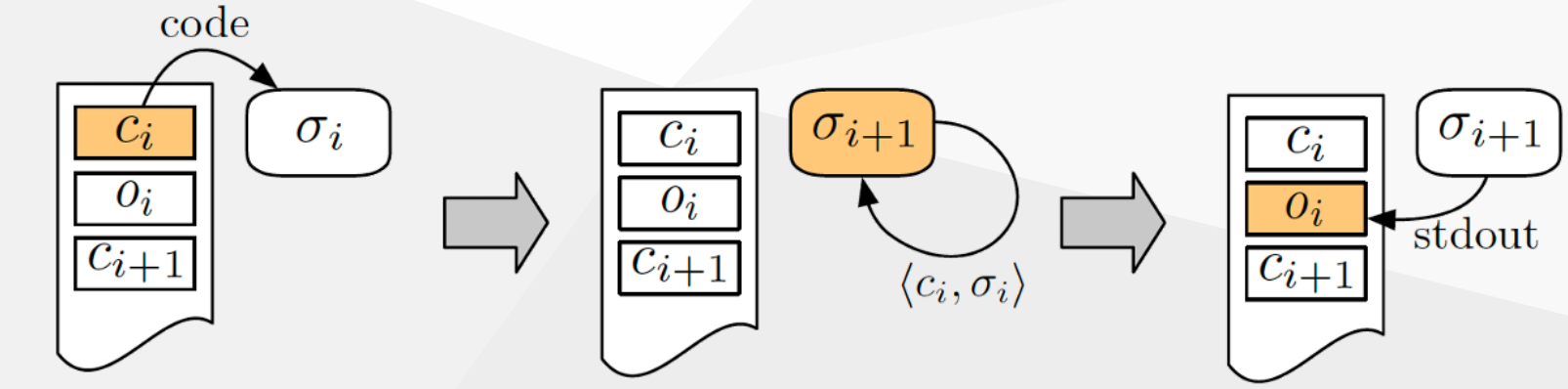
I. Colonnelli, D. Medic, A. Mulone, V. Bono, L. Padovani, and M. Aldinucci, "Introducing SWIRL: An Intermediate Representation Language for Scientific Workflows," in Formal Methods - 26th International Symposium, FM 2024, Milan, Italy, September 9-13, 2024, Proceedings, Part I, 2024, vol. 14933, pp. 226–244. doi:10.1007/978-3-031-71162-6_12

Literate computing

A **computational notebook** can be seen as an ordered list of **cells**, each containing code, code executions output, or natural language documentation

Code cells are treated as the **atomic execution units** of a notebook. Each cell is executed in a **state**, a partial mapping from host language identifiers to first-class objects

A dedicated **driver process** (e.g., the kernel in Jupyter Notebooks or the interpreter in Apache Zeppelin) manages that state, guaranteeing a **unique total order** of cells executions

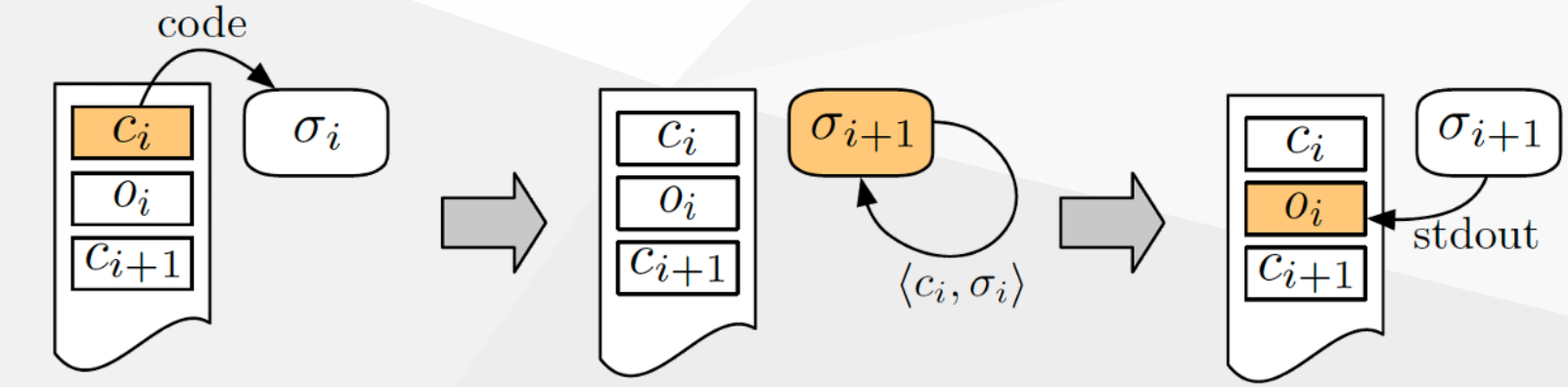


Literate computing: limitations

Notebooks' **purely sequential** execution flow makes it impossible to exploit the inherent concurrency of workflow graphs

The lack of a rigorous workflow model prevents to satisfy non-functional requirements like **portability**, **reproducibility**, and **provenance**

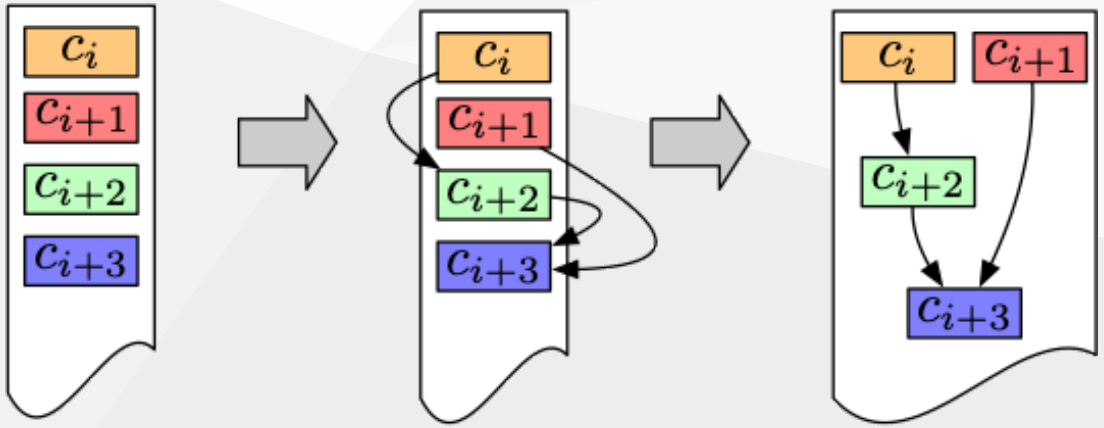
Using Notebooks as a high-level interface to support runtime intervention in the Continuum is challenging due to the lack of support for **hybrid distributed topologies** of execution kernels



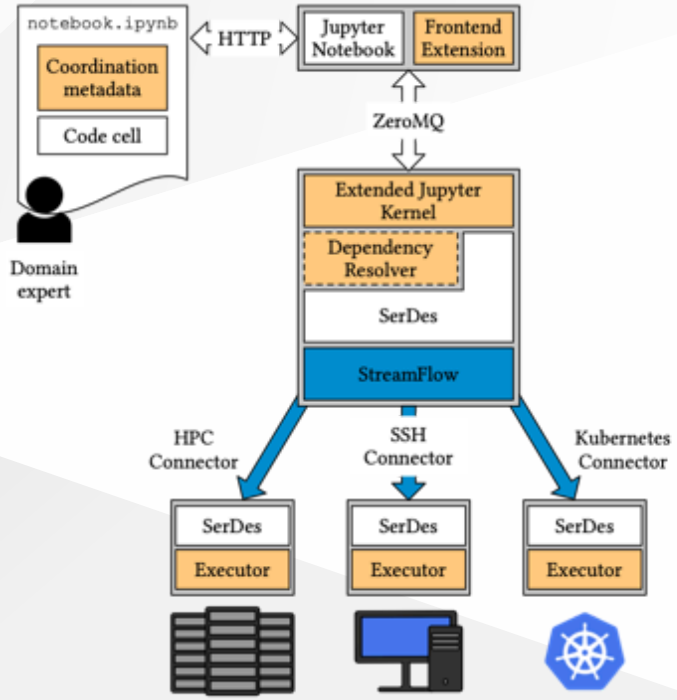
Hybrid literate workflows

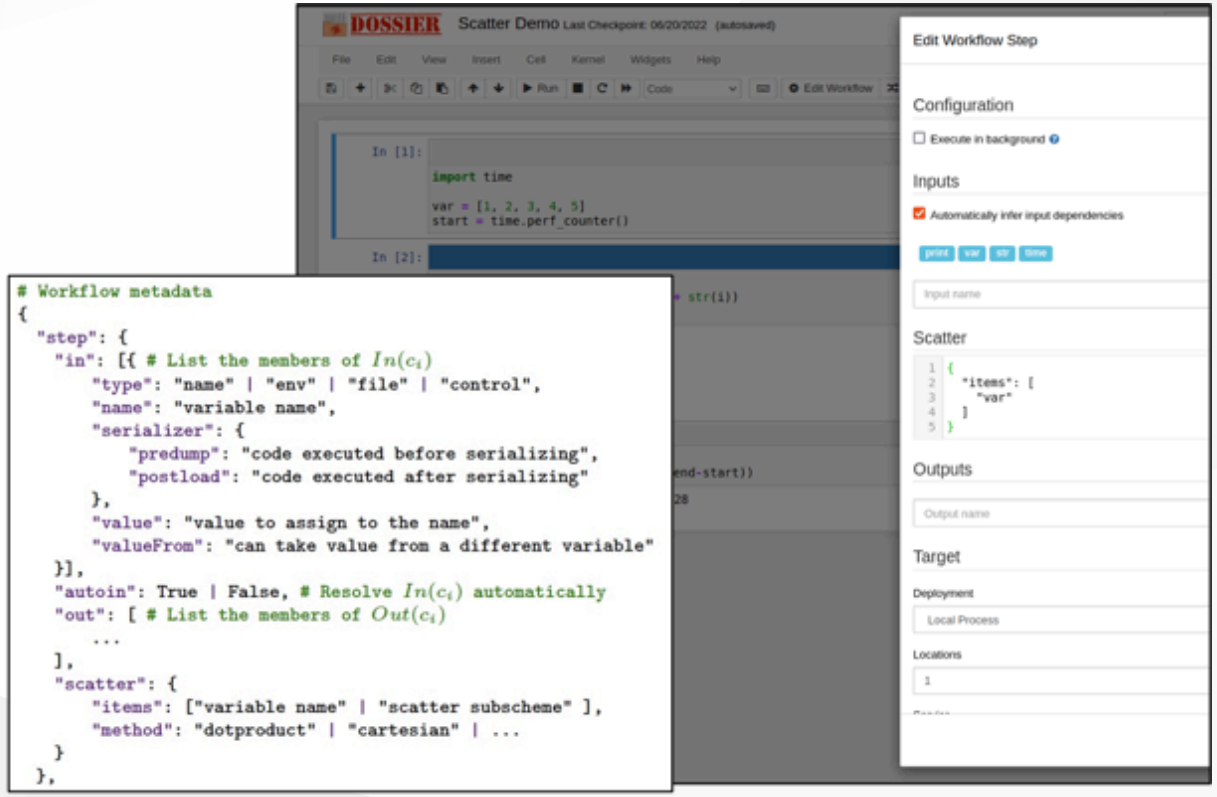
Treat a computational notebook as a **hybrid interactive workflow**:

- Map each notebook cell into a **workflow task**
- Infer inter-cell (true) **data dependencies** to construct a distributed workflow grpah
- Derive **sequentially equivalent** parallel semantics to allow concurrent execution of code cells without impacting the progam correctness
- Extend the Notebook **metadata format** to describe **topologies** of deployment locations, **mapping** relations, and explicit intra-cell **data-parallel constructs** (e.g. scatter/gather)



Jupyter Workflow






```
# Workflow metadata
{
  "step": {
    "in": [{ # List the members of In(c_i)
        "type": "name" | "env" | "file" | "control",
        "name": "variable name",
        "serializer": {
            "predump": "code executed before serializing",
            "postload": "code executed after serializing"
        },
        "value": "value to assign to the name",
        "valueFrom": "can take value from a different variable"
    }],
    "autoin": True | False, # Resolve In(c_i) automatically
    "out": [ # List the members of Out(c_i)
        ...
    ],
    "scatter": {
        "items": ["variable name" | "scatter subscheme" ],
        "method": "dotproduct" | "cartesian" | ...
    }
  },
```

JUPYTER WORKFLOW

https://jupyter-workflow.di.unito.it

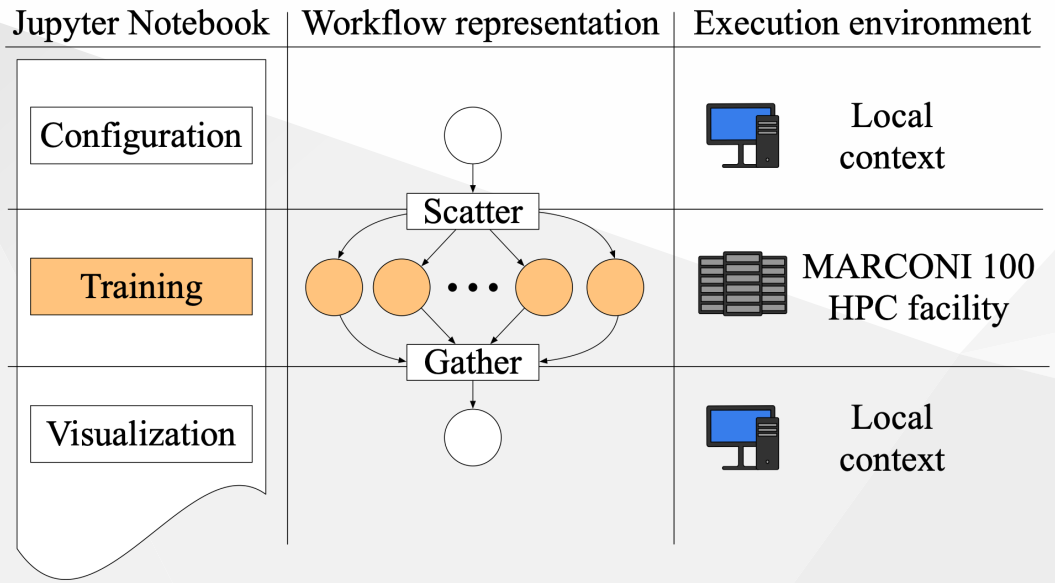
I. Colonnelli, M. Aldinucci, B. Cantalupo, L. Padovani, S. Rabellino, C. Spampinato, R. Morelli, R. Di Carlo, N. Magini, and C. Cavazzoni, "Distributed workflows with Jupyter," Future generation computer systems, vol. 128, pp. 282-298, 2022. doi:10.1016/j.future.2021.10.007

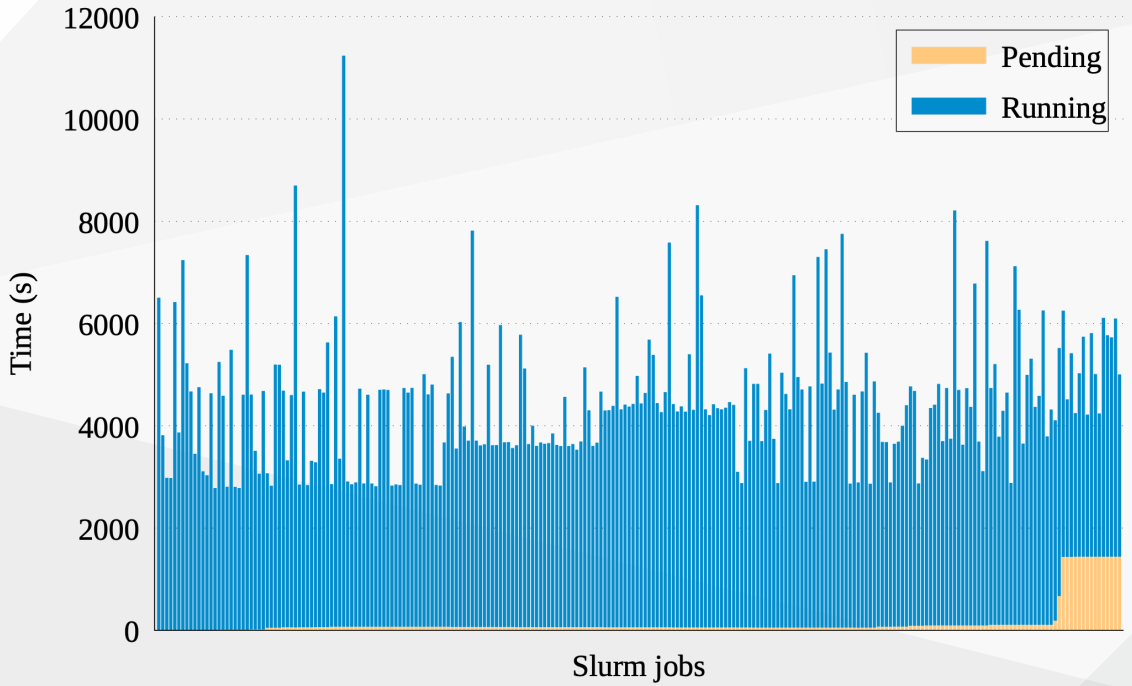
Case study: CLAIRE COVID-19 pipeline

UNIVERSITÀ DI TORINO

240 instances of a DenseNet **hyperparameter search** workflow running in parallel on 240 GPU-equipped nodes of the CINECA MARCONI 100 facility (NVIDIA v100)

Obtained speedup: **92x** (3h vs. 288h on a single V100 GPU)





Data transport: file-based vs. streaming

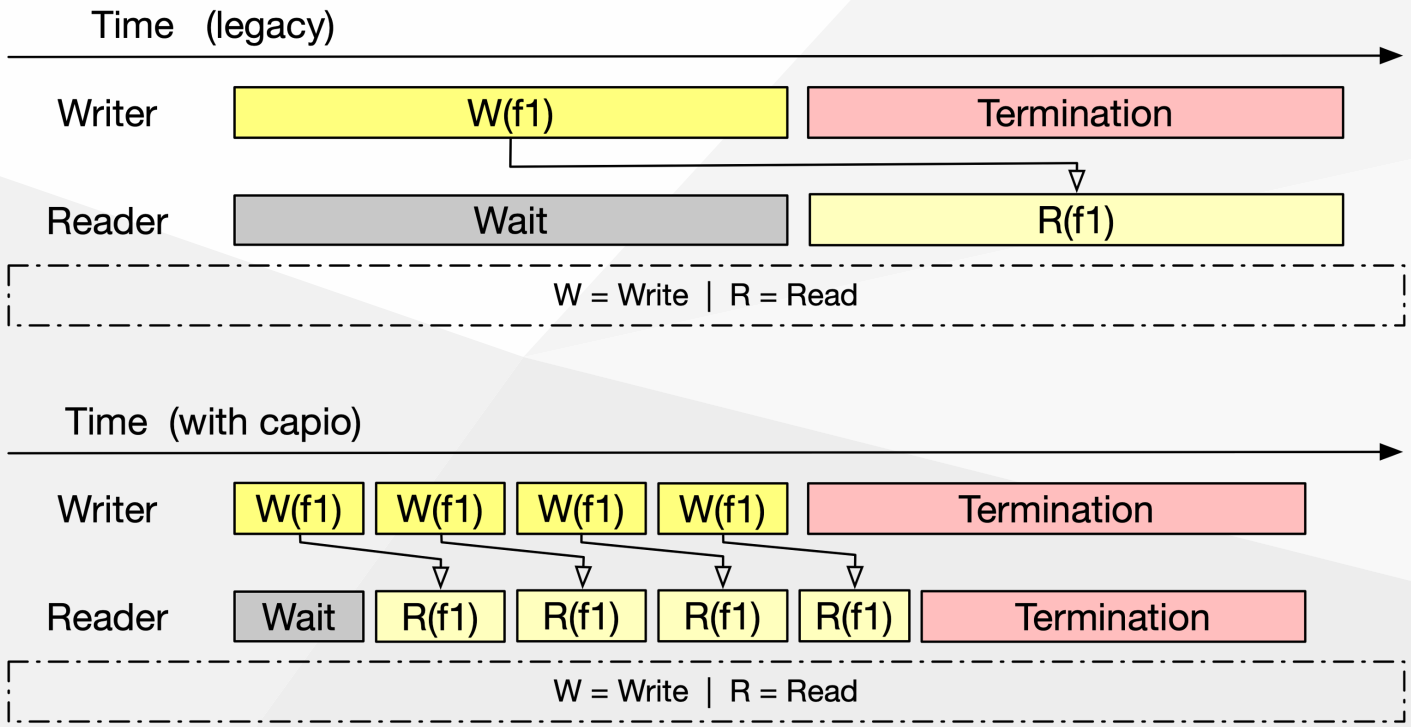
Standard scientific workflows adopt a **post-hoc execution model**: a step can start when all the steps it depends on terminate, saving their output data in a persistent storage

WMSs assume that there exists a **common storage system** shared among all involved locations (e.g., a parallel FS in HPC, or an S3 object storage in Cloud)

In Big Data applications, data are sent from Edge sensors to Cloud consumers (e.g., Apche Spark) through **streaming message brokers** (e.g., Apache Kafka, Spark Streaming, ...)

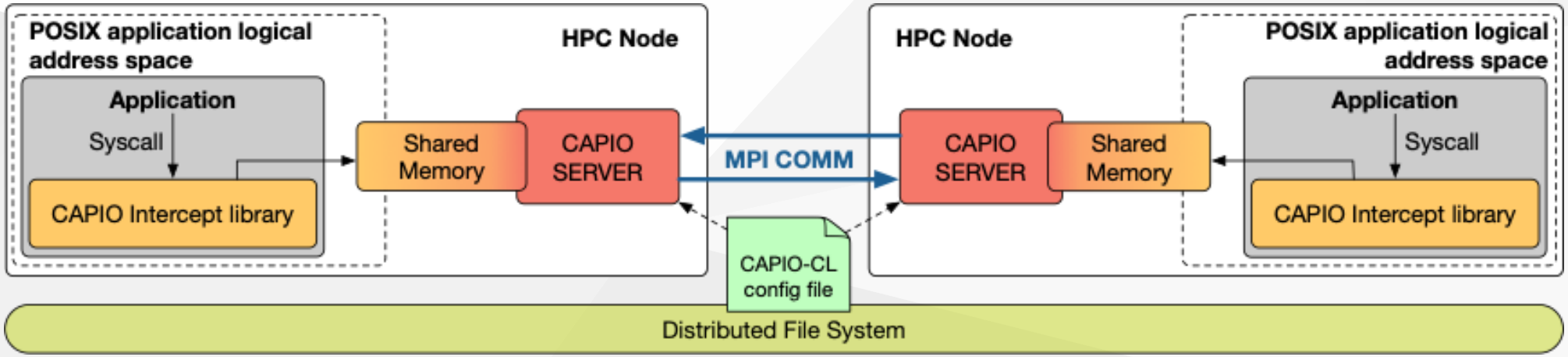
Transparent streaming workflows

A file-based workflow can be **transparently converted** into a streaming workflow (without changing the steps' internal code) by **co-scheduling** subsequent steps and ensuring dependencies through **read/write communication primitives**



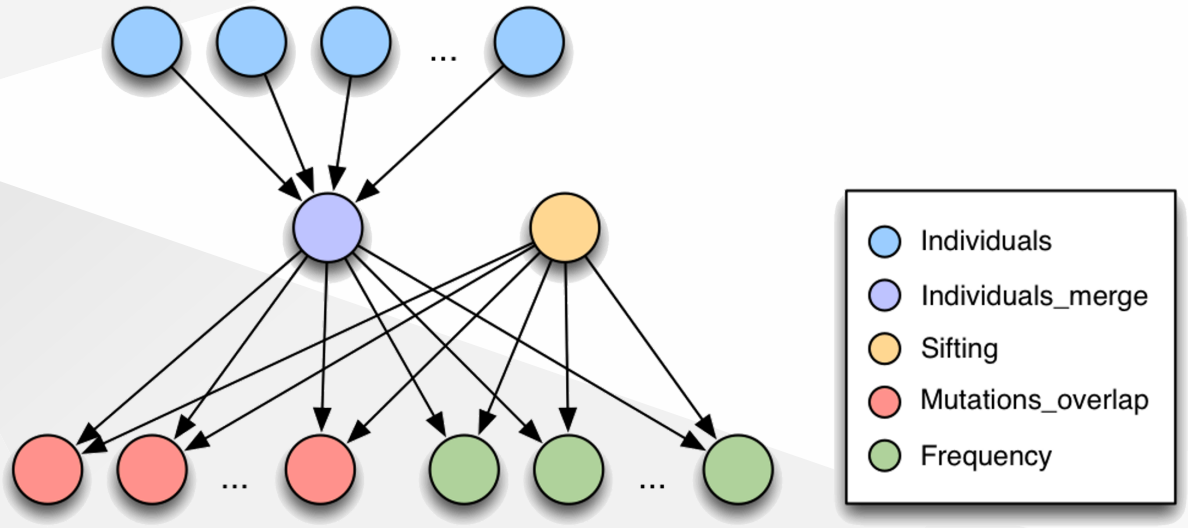
The CAPIO midleware

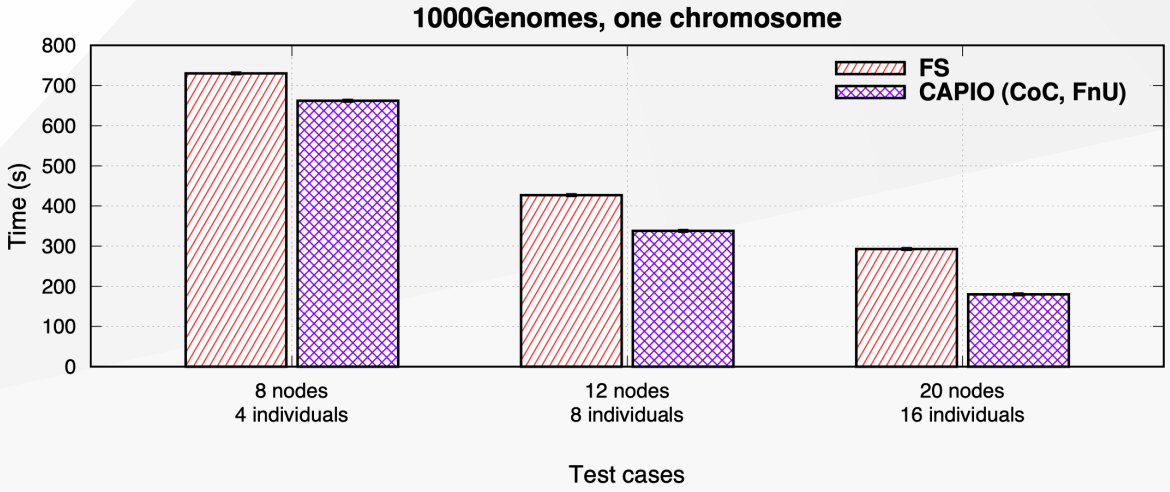
The **Cross-Application Programmable I/O (CAPIO)** middleware implements transparent streaming workflows by intercepting **POSIX read/write syscalls** and moving data from porducers to consumers using a **configurable communication backend** (e.g., MPI, file-system, MQTT)



A. R. Martinelli, M. Torquati, M. Aldinucci, I. Colonnelli and B. Cantalupo, "CAPIO: a Middleware for Transparent I/O Streaming in Data- Intensive Workflows," 2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics (HiPC), Goa, India, 2023, pp. 153-163. doi:10.1109/HiPC58850.2023.00031

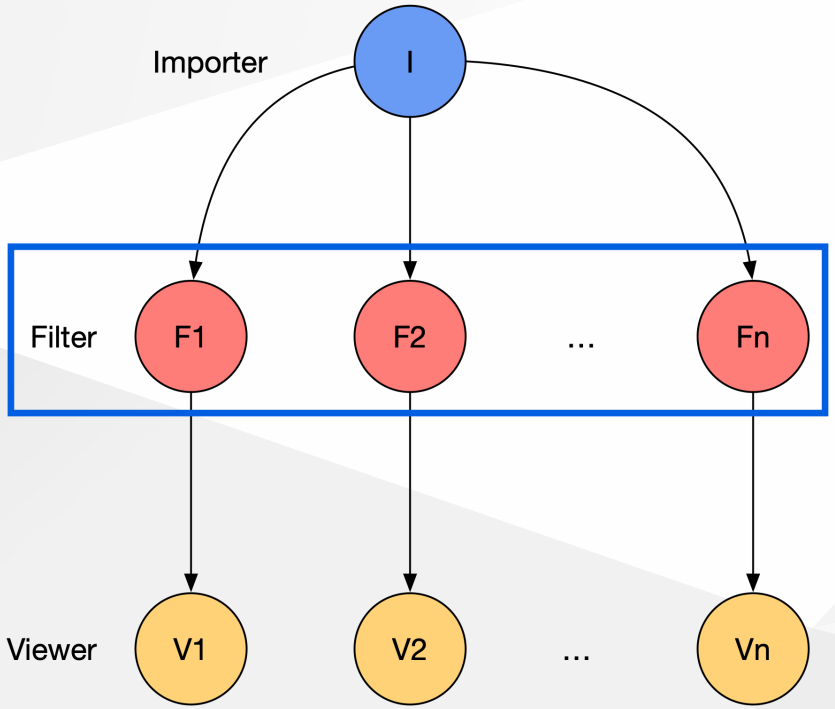
Case study: 1000 Genome wokflow

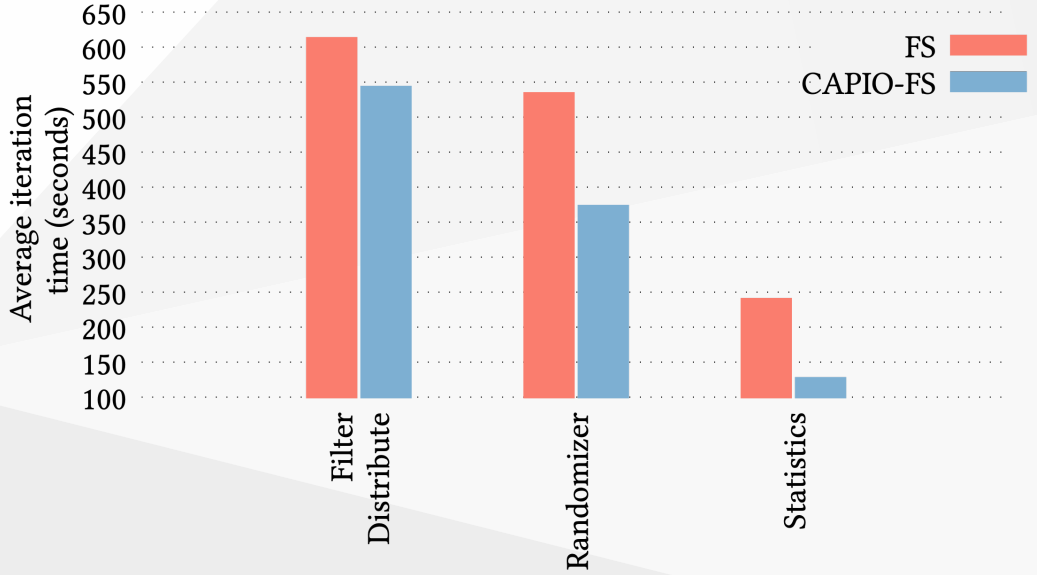




A. R. Martinelli, M. Torquati, M. Aldinucci, I. Colonnelli and B. Cantalupo, "CAPIO: a Middleware for Transparent I/O Streaming in Data- Intensive Workflows," 2023 IEEE 30th International Conference on High Performance Computing, Data, and Analytics (HiPC), Goa, India, 2023, pp. 153-163. doi:10.1109/HiPC58850.2023.00031

Case study: VisIVO workflow





M. E. Santimaria, I. Colonnelli, B. Cantalupo, M. Torquati, N. Tuccari, E. Sciacca, and M. Aldinucci, "Dynamic transparent streaming in file-based workflows with CAPIO," Future Generation Computer Systems. (submitted)